

# Applied Machine Learning

## Introduction to Neural Networks



UNIVERSITY OF  
GOTHENBURG

---

**CHALMERS**

Why neural networks?

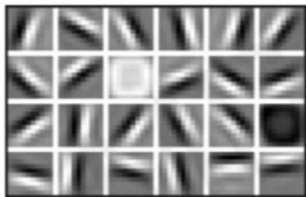
And why now?

# Why neural networks

Hand engineered features are time consuming, brittle, and not scalable in practice

Can we learn the **underlying features** directly from data?

Low Level Features



Lines & Edges

Mid Level Features



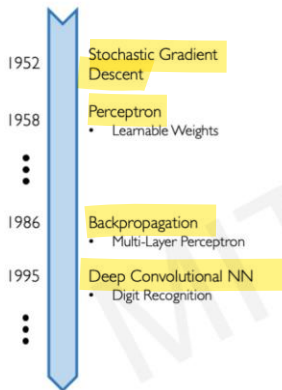
Eyes & Nose & Ears

High Level Features



Facial Structure

# Why now



Neural Networks date back decades, so why the resurgence?

## 1. Big Data

- Larger Datasets
- Easier Collection & Storage

IMAGENET



## 2. Hardware

- Graphics Processing Units (GPUs)
- Massively Parallelizable



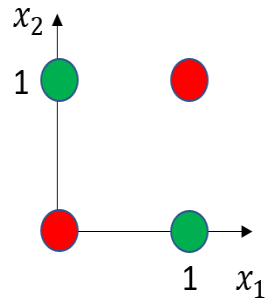
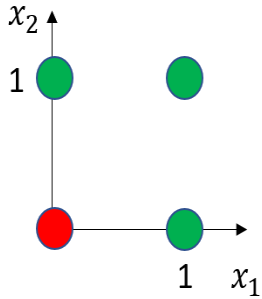
## 3. Software

- Improved Techniques
- New Models
- Toolboxes



# prelude: getting rid of linear inseparability

example: XOR dataset



## example: XOR dataset

```
import numpy
from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score

X = numpy.array([[1, 1],
                  [1, 0],
                  [0, 1],
                  [0, 0]])

Y = ['no', 'yes', 'yes', 'no']

clf = LinearSVC()
clf.fit(X, Y)
print(accuracy_score(Y, clf.predict(X)))
```

0.5

## example: XOR dataset with a combination feature

```
import numpy
from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score

# feature1, feature2, feature1&feature2
X = numpy.array([[1, 1, 1],
                 [1, 0, 0],
                 [0, 1, 0],
                 [0, 0, 0]])

Y = ['no', 'yes', 'yes', 'no']

# now we have linear separability, so we get 100%
clf = LinearSVC()
clf.fit(X, Y)
print(accuracy_score(Y, clf.predict(X)))
```

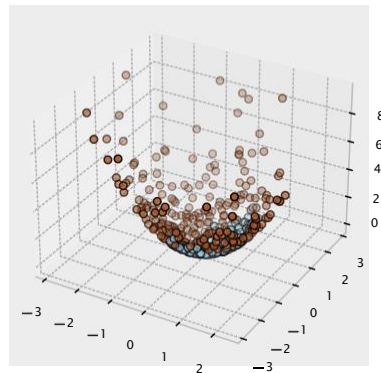
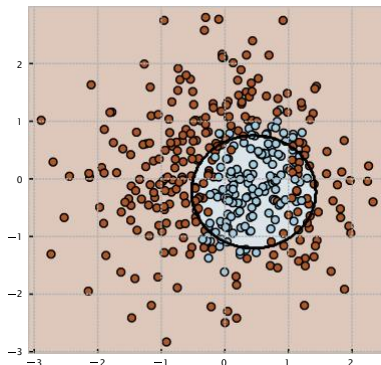
1.0

## another example

- ▶ in the other example, we may introduce a **third feature**

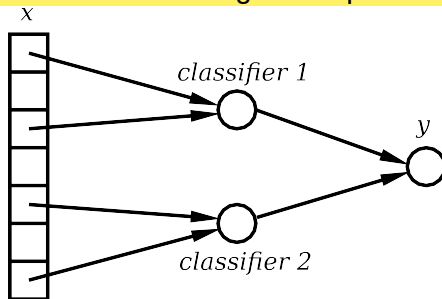
$$x_3 = x_1^2 + x_2^2$$

- ▶ by applying this **nonlinear transformation**, we move into a space where the classes are more or less linearly separable



## expressing feature combinations as “sub-classifiers”

- ▶ instead of hand-crafting additional features, such as  $x_3 = x_1 \& x_2$ , we could imagine that the combination feature  $x_3$  would be computed by a separate classifier, for instance LR
- ▶ we could train a classifier using the output of “sub-classifiers”



## a multilayered model

- ▶ a **feedforward neural network** or **multilayer perceptron** consists of **connected layers** of “submodels”
  - ▶ the intermediate steps are called **hidden units**
  - ▶ the final classifier or regressor is called the **output unit**
- ▶ each hidden unit  $h_i$  computes its output based on its own weight vector  $w_{h_i}$ :

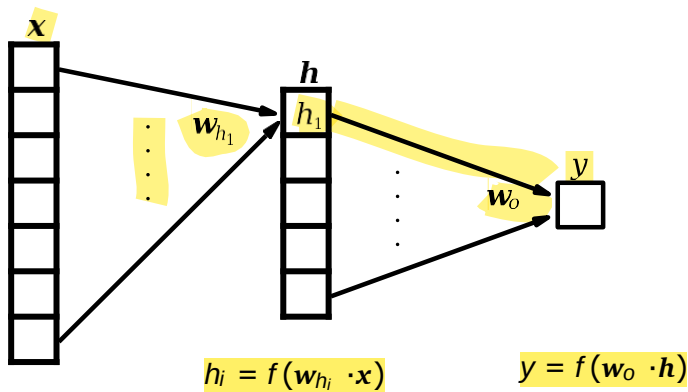
$$h_i = f(w_{h_i} \cdot x)$$

- ▶ and then the **output** is **computed** from the hidden units:

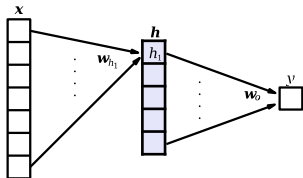
$$y = f(w_o \cdot h)$$

- ▶ the **function  $f$**  is called the **activation**
  - ▶ for now, let's assume that  $f$  is the sigmoid function, so the hidden units and output unit can be seen as LR classifiers

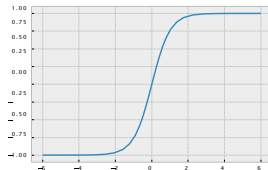
## two-layered feedforward NN: illustration



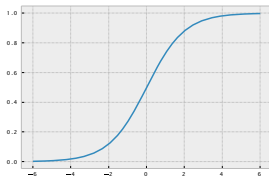
# common activation functions: hidden layers



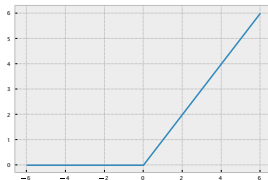
► **tanh** (hyperbolic tangent):



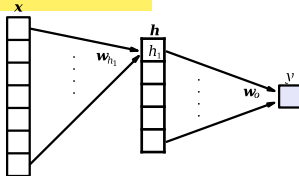
► **sigmoid** (logistic):



► **ReLU** (rectified linear unit):

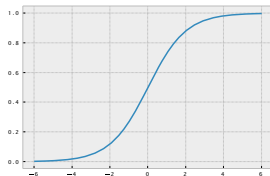


## output layer: binary classification



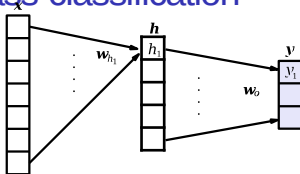
- **sigmoid** (logistic) for a **binary** classifier:

$$P(\text{positive}|x) = \frac{1}{1 + e^{-\text{score}}}$$



- When the value is **large positive** → *sigmoid score* close to 1
- Produces values (real numbers) in the **range [0..1]**
- How to **map from probabilities to two outputs**? Use **rounding** (usually  $\geq 0.5$  set to 1, otherwise to 0)

## output layer: multiclass classification

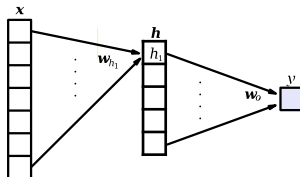


- **softmax** for a **multiclass** classifier (that is, more than 2 output classes):

$$P(\text{output} = \text{class } i | x) = \frac{e^{\text{score}_i}}{\sum_k e^{\text{score}_k}}$$

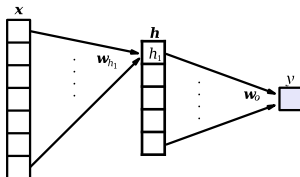
Given a vector of  $K$  real values (can be positive, negative, or 0), **softmax** function produces a vector of  $K$  real values (between 0 and 1) that all sum up to 1. Therefore, the outputs can be considered as probabilities of class membership.

## output layer: regression



- what activation should we use for a **regression** problem?

## output layer: regression



- ▶ what activation should we use for a **regression** problem?
- ▶ typically, **no activation function is used here**
  - ▶ so this is a **linear regression model on top of the hidden layer**

# Discussion

We use NN to classify scan images into benign vs malignant.  
What activation function should we use for the output layer and why?

# Discussion

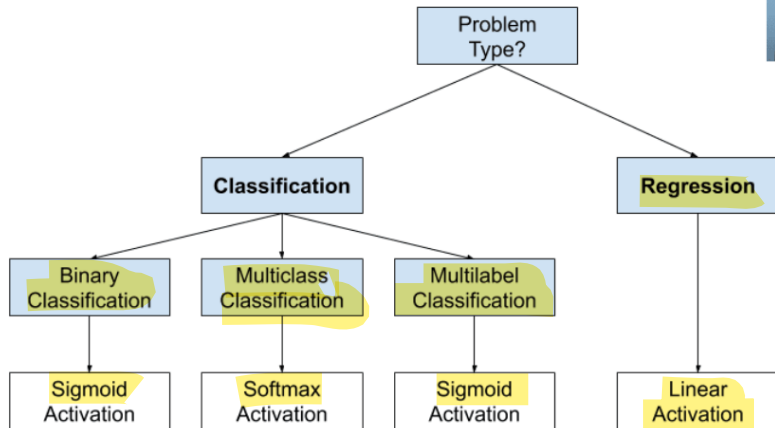
We use NN for classifying digit images to either 0,1,2,... or 9.  
What activation function should we use for the output layer and why?

# Summary activation function

- Different layers (e.g., hidden layers vs output layer) can use different activation functions.
- Same activation function is used in all units in the same layer. Usually, the same activation function is used in all units of hidden layers.
- What function to use in the hidden layers is based on the type of NN architecture (e.g., CNN, RNN) that is used.

# Summary activation function

## How to Choose an Output Layer Activation Function



MachineLearningMastery.com

How to Choose an Output Layer Activation Function

Notebook examples:

Nonlinear regression toy example

Nonlinear classification toy example

# expressivity of feedforward NNs

- ▶ Hornik's **universal approximation theorem** shows that feedforward NNs can approximate any (bounded) mathematical function
  - ▶ Hornik (1991). [Approximation capabilities of multilayer feedforward networks](#). Neural Networks, 4(2), 251–257.
- ▶ and this is true even with a single hidden layer!
- ▶ however, this is mainly of theoretical interest
  - ▶ the theorem does not say how many hidden units we need
  - ▶ and it doesn't say how the network should be trained
- ▶ although a single hidden layer is sufficient in theory, in practice it can be better to have several hidden layers

# Training neural networks

# training feedforward neural networks

- ▶ training a NN consists of finding the weights in the layers
- ▶ so how do we find those weights?

# training feedforward neural networks

- ▶ training a NN consists of finding the weights in the layers
- ▶ so how do we find those weights?
- ▶ as we did for the SVC and LR!
  - ▶ state an **objective function** including a **loss** and possibly a **regularizer**
  - ▶ apply an **optimization algorithm** to find the weights that minimize the objective

## SGD: pseudocode

initialize  $w$

repeat ...

    pick a training instance  $(x_i, y_i)$

    compute gradient  $\nabla f_i$  of the loss for current instance  $(x_i, y_i)$

$w = w - \eta \cdot \nabla f_i(w)$

return  $w$

# loss functions

- ▶ **log loss** for a **binary classifier**

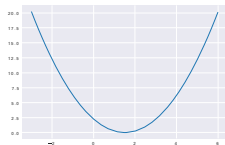
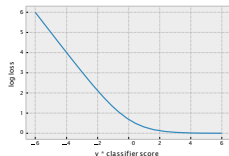
$$\text{Loss} = -\log(P(\text{output} = y))$$

- ▶ **cross-entropy loss** for **multiclass**

$$\text{Loss} = -\log(P(\text{output} = y))$$

- ▶ **squared error loss** for **regression**

$$\text{Loss} = (y - o)^2$$

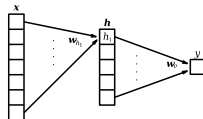


## example

- ▶ let's use two layers with sigmoid units, and then the log loss

$$h = \sigma(W_h \cdot x)$$
$$y = \sigma(W_o \cdot h)$$

$$\text{Loss} = -\log(y)$$



- ▶ so the whole thing becomes

$$\text{Loss} = -\log \sigma(W_o \cdot \sigma(W_h \cdot x))$$

- ▶ now, to do gradient descent, we need to compute gradients w.r.t. the weights  $W_h$  and  $W_o$
- ▶ **ouch!** it looks completely unwieldy!

# the chain rule of derivatives/gradients

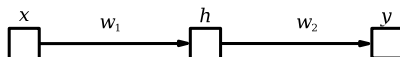
- ▶ NNs consist of functions applied to the output of other functions

$$\text{Loss} = -\log \sigma(W_o \cdot \sigma(W_h \cdot x))$$

- ▶ in calculus, we use the **chain rule** to compute gradients of nested functions
  - ▶ assume that we apply the function  $f$  to the output of  $g$
  - ▶ then the chain rule says how we can compute the gradient of the combination:

$$\text{gradient of } f(g(x)) = \text{gradient of } f(g) \cdot \text{gradient of } g(x)$$

## chain rule example

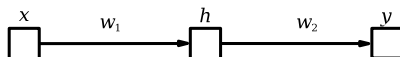


- ▶ let's say we have defined a simple neural network:

$$h = f_1(w_1 \cdot x)$$

$$\text{Loss} = f_2(w_2 \cdot h)$$

## chain rule example



- ▶ let's say we have defined a simple neural network:

$$h = f_1(w_1 \cdot x)$$

$$\text{Loss} = f_2(w_2 \cdot h)$$

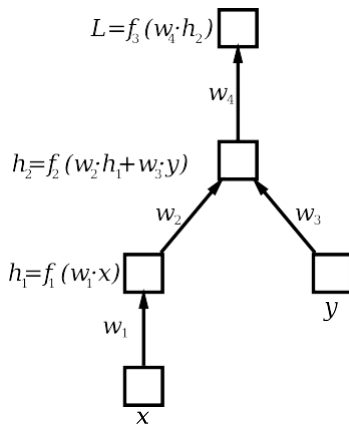
gradient of  $f(g(x)) = \text{gradient of } f(g) \cdot \text{gradient of } g(x)$

- ▶ then we can compute the gradients with respect to  $w_1$  and  $w_2$  using the chain rule:

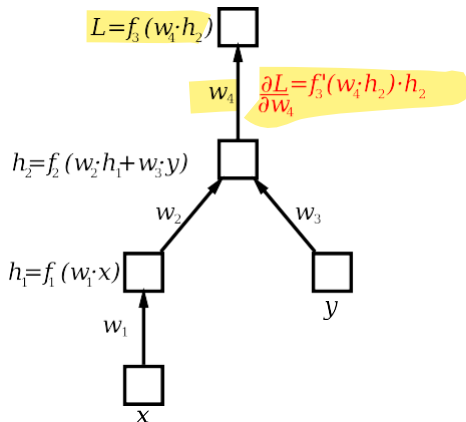
$$\frac{\partial \text{Loss}}{\partial w_2} = f'_2(w_2 \cdot h) \cdot h$$

$$\frac{\partial \text{Loss}}{\partial w_1} = f'_2(w_2 \cdot h) \cdot w_2 \cdot f'_1(w_1 \cdot x) \cdot x$$

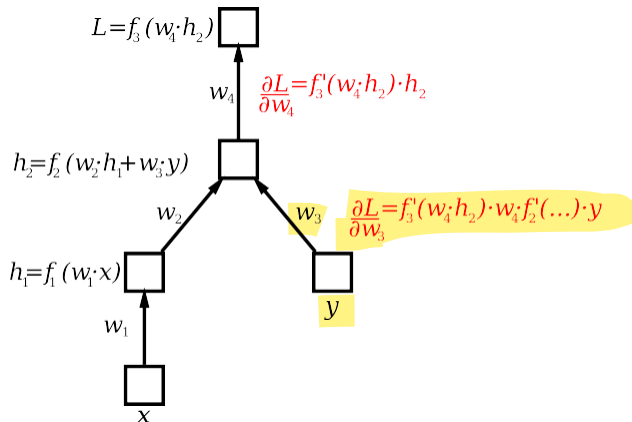
## slightly more complicated computational graph



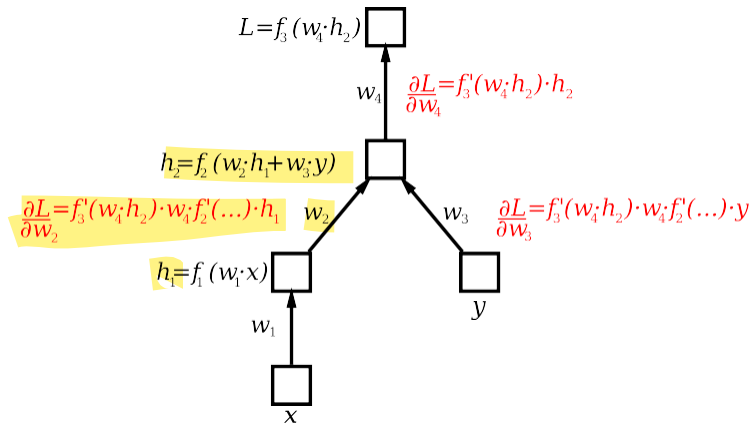
## slightly more complicated computational graph



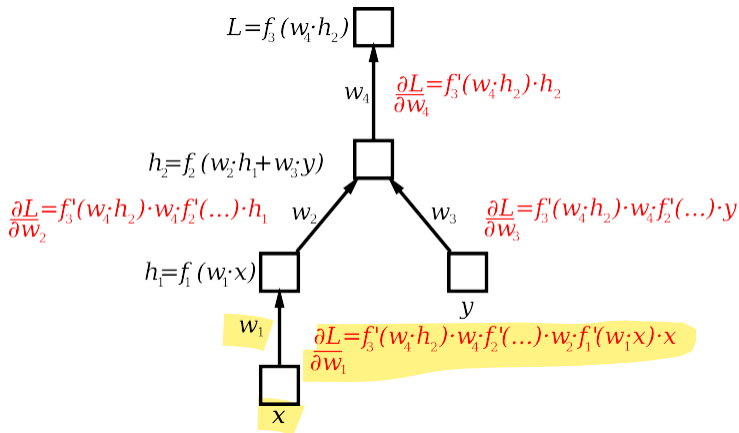
## slightly more complicated computational graph



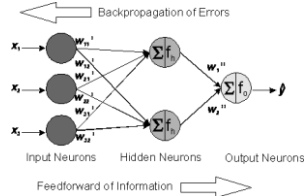
## slightly more complicated computational graph



# slightly more complicated computational graph



## the general recipe: backpropagation



- ▶ using the chain rule, the gradients of the weights in each layer can be computed from the gradients of the layers after it
- ▶ this trick is called **backpropagation**
- ▶ the general form of backprop is a bit kludgy and involves a lot of matrix book-keeping
- ▶ fortunately, there are computer programs that can do the algebra for us!
  - ▶ in NN software, we usually just declare the network and the loss, then the gradients are computed under the hood

# neural network training: summary

- ▶ the key idea is the same as for linear models: we define an **objective** and we set the weights to **optimize** it
  - ▶ e.g. the **cross-entropy** loss optimized with **SGD**
- ▶ optimization requires that we compute **gradients**
- ▶ **gradients** are computed by the **backpropagation** algorithm
  - ▶ backprop is based on the **chain rule**

# Applied Machine Learning

## Neural Networks: Practical Aspects



UNIVERSITY OF  
GOTHENBURG

---

**CHALMERS**

## overview of neural network libraries

# training efficiency of NNs

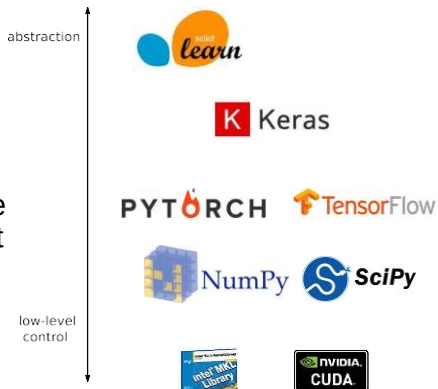
- ▶ our previous classifiers took seconds or minutes to train
- ▶ NNs tend to take minutes, hours, days, weeks ...
  - ▶ depending on the complexity of the network and the amount of training data
- ▶ NNs use a lot of linear algebra (matrix multiplications) so it can be useful to work to speed up the math
  - ▶ parallelize as much as possible
  - ▶ use optimized math libraries
  - ▶ use a GPU
  - ▶ no need to implement backprop
  - ▶ in short: you're better off using a specialized NN library

# neural network software: Python

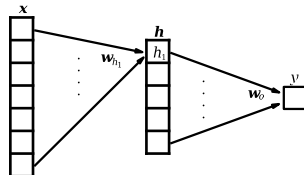
- ▶ scikit-learn currently has quite limited support for NNs
- ▶ the main NN software in the Python world used to be **Theano**
  - ▶ developed by Yoshua Bengio's group in Montréal
  - ▶ <http://deeplearning.net/software/theano>
- ▶ the major players have released their own libraries in the last few years:
  - ▶ Google: [TensorFlow](#)
  - ▶ Facebook: [PyTorch](#)
  - ▶ Microsoft: [CNTK](#)
- ▶ these toolkits provide “building blocks” such as layers, activations, losses, regularizers, optimizers, ...

# neural network software: Python (2)

- ▶ TensorFlow etc. do a lot of useful math stuff, and integrate nicely with the GPU, but they can be a bit low-level
- ▶ so there are libraries that create a more high-level interface, a bit similar to scikit-learn
  - ▶ Keras



# coding example with Keras



```
keras_model = Sequential()

n_hidden = 3
keras_model.add(Dense(input_dim=X.shape[1],
                       output_dim=n_hidden))
keras_model.add(Activation("sigmoid"))

keras_model.add(Dense(input_dim=n_hidden,
                       output_dim=1))
keras_model.add(Activation("sigmoid"))

keras_model.compile(loss='binary_crossentropy',
                    optimizer='sgd')

keras_model.fit(X, Y)
```

tricks of the trade

# feature preprocessing

- ▶ NNs are very sensitive to feature preprocessing

# feature preprocessing: example

Accuracy  
79,30%

| RowNumber | CustomerId | Surname   | CreditScore | Geography | Gender | Age | Tenure | Balance | NumOfProducts | HasCrCard |   |         |   |
|-----------|------------|-----------|-------------|-----------|--------|-----|--------|---------|---------------|-----------|---|---------|---|
| 1         | 15634682   | Hargrave  | 619         | France    | Female | 42  | 2      | 0       | 1             | 1         | 1 | 101349  | 1 |
| 2         | 15647311   | Hill      | 608         | Spain     | Female | 41  | 1      | 83807.9 | 1             | 0         | 1 | 112543  | 0 |
| 3         | 15619384   | Onio      | 502         | France    | Female | 42  | 8      | 159661  | 3             | 1         | 0 | 113932  | 1 |
| 4         | 15701354   | Boni      | 699         | France    | Female | 39  | 1      | 0       | 2             | 0         | 0 | 93826.6 | 0 |
| 5         | 15737888   | Mitchell  | 850         | Spain     | Female | 43  | 2      | 125511  | 1             | 1         | 1 | 79004.1 | 0 |
| 6         | 15574012   | Chu       | 645         | Spain     | Male   | 44  | 8      | 113756  | 2             | 1         | 0 | 149757  | 1 |
| 7         | 15592531   | Bartlett  | 822         | France    | Male   | 50  | 7      | 0       | 2             | 1         | 1 | 10062.8 | 0 |
| 8         | 15656148   | Obinna    | 376         | Germany   | Female | 29  | 4      | 115047  | 4             | 1         | 0 | 119347  | 1 |
| 9         | 15792365   | He        | 501         | France    | Male   | 44  | 4      | 142051  | 2             | 0         | 1 | 74940.5 | 0 |
| 10        | 15592389   | H7        | 684         | France    | Male   | 27  | 2      | 134604  | 1             | 1         | 1 | 71725.7 | 0 |
| 11        | 15767821   | Bearce    | 528         | France    | Male   | 31  | 6      | 102017  | 2             | 0         | 0 | 80181.1 | 0 |
| 12        | 15737173   | Andrews   | 497         | Spain     | Male   | 24  | 3      | 0       | 2             | 1         | 0 | 76390   | 0 |
| 13        | 15632264   | Kay       | 476         | France    | Female | 34  | 10     | 0       | 2             | 1         | 0 | 26261   | 0 |
| 14        | 15691483   | Chin      | 549         | France    | Female | 25  | 5      | 0       | 2             | 0         | 0 | 190858  | 0 |
| 15        | 15600882   | Scott     | 635         | Spain     | Female | 35  | 7      | 0       | 2             | 1         | 1 | 65951.6 | 0 |
| 16        | 15643966   | Goforth   | 616         | Germany   | Male   | 45  | 3      | 143129  | 2             | 0         | 1 | 64327.3 | 0 |
| 17        | 15737452   | Romeo     | 653         | Germany   | Male   | 58  | 1      | 132603  | 1             | 1         | 0 | 5097.67 | 1 |
| 18        | 15788218   | Henderson | 549         | Spain     | Female | 24  | 9      | 0       | 2             | 1         | 1 | 14406.4 | 0 |
| 19        | 15661507   | Muldrow   | 587         | Spain     | Male   | 45  | 6      | 0       | 1             | 0         | 0 | 150685  | 0 |
| 20        | 15560982   | Hao       | 726         | France    | Female | 24  | 6      | 0       | 2             | 1         | 1 | 54724   | 0 |

► [sklearn.neural\\_network.MLPClassifier](#)

<https://medium.com/@piyush.kailash.chaudhari/importance-of-feature-scaling-for-artificial-neural-networks-and-k-nearest-neighbors-4b7aa618d5ea>

# feature preprocessing: example

Accuracy  
86.10%

| RowNumber | CustomerId | Surname   | CreditScore | Geography | Gender | Age        | Tenure      | Balance  | NumOfProducts | HasCrCard | IsActiveMember | EstimatedSalary | Exited |
|-----------|------------|-----------|-------------|-----------|--------|------------|-------------|----------|---------------|-----------|----------------|-----------------|--------|
| 1         | 15634682   | Hargrave  | -0.326221   | France    | Female | 0.293517   | -1.04176    | -1.22585 | 1             | 1         | 1              | 0.0218865       | 1      |
| 2         | 15647311   | Hill      | -0.440036   | Spain     | Female | 0.198164   | -1.38754    | 0.11735  | 1             | 0         | 1              | 0.216534        | 0      |
| 3         | 15619384   | Onio      | -1.53679    | France    | Female | 0.293517   | 1.03291     | 1.33385  | 3             | 1         | 0              | 0.248687        | 1      |
| 4         | 15701354   | Boni      | 0.501521    | France    | Female | 0.00745665 | -1.38754    | -1.22585 | 2             | 0         | 0              | -0.100918       | 0      |
| 5         | 15737888   | Mitchell  | 2.06388     | Spain     | Female | 0.388871   | -1.04176    | 0.785728 | 1             | 1         | 1              | -0.365276       | 0      |
| 6         | 15574012   | Chu       | -0.0572053  | Spain     | Male   | 0.484225   | 1.03291     | 0.597329 | 2             | 1         | 0              | 0.86365         | 1      |
| 7         | 15592531   | Bartlett  | 1.77417     | France    | Male   | 1.05635    | 0.68713     | -1.22585 | 2             | 1         | 1              | -1.56549        | 0      |
| 8         | 15656148   | Obinna    | -2.84049    | Germany   | Female | -0.946079  | -0.350204   | 0.610019 | 4             | 1         | 0              | 0.334854        | 1      |
| 9         | 15792365   | He        | -1.54714    | France    | Male   | 0.484225   | -0.350204   | 1.05082  | 2             | 0         | 1              | -0.437329       | 0      |
| 10        | 15592389   | H?        | 0.346319    | France    | Male   | -1.13679   | -1.04176    | 0.931463 | 1             | 1         | 1              | -0.49323        | 0      |
| 11        | 15767821   | Bearce    | -1.26778    | France    | Male   | -0.755372  | 0.341352    | 0.409185 | 2             | 0         | 0              | -0.3462         | 0      |
| 12        | 15737173   | Andrews   | -1.58853    | Spain     | Male   | -1.42285   | -0.695982   | -1.22585 | 2             | 1         | 0              | -0.412123       | 0      |
| 13        | 15632264   | Kay       | -1.80581    | France    | Female | -0.469311  | 1.72446     | -1.22585 | 2             | 1         | 0              | -1.28382        | 0      |
| 14        | 15691483   | Chin      | -1.0505     | France    | Female | -1.32749   | -0.00442596 | -1.22585 | 2             | 0         | 0              | 1.57836         | 0      |
| 15        | 15600882   | Scott     | -0.160673   | Spain     | Female | -0.373958  | 0.68713     | -1.22585 | 2             | 1         | 1              | -0.593636       | 0      |
| 16        | 15643966   | Goforth   | -0.357262   | Germany   | Male   | 0.579578   | -0.695982   | 1.0681   | 2             | 0         | 1              | -0.621882       | 0      |
| 17        | 15737452   | Romeo     | 0.025569    | Germany   | Male   | 1.81917    | -1.38754    | 0.899393 | 1             | 1         | 0              | -1.65183        | 1      |
| 18        | 15788218   | Henderson | -1.0505     | Spain     | Female | -1.42285   | 1.37869     | -1.22585 | 2             | 1         | 1              | -1.48996        | 0      |
| 19        | 15661507   | Muldrow   | -0.657318   | Spain     | Male   | 0.579578   | 0.341352    | -1.22585 | 1             | 0         | 0              | 1.0189          | 0      |
| 20        | 15568982   | Hao       | 0.780884    | France    | Female | -1.42285   | 0.341352    | -1.22585 | 2             | 1         | 1              | -0.788873       | 0      |

► [sklearn.neural\\_network.MLPClassifier](#)

► <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html>

<https://medium.com/@piyush.kailash.chaudhari/importance-of-feature-scaling-for-artificial-neural-networks-and-k-nearest-neighbors-4b7aa618d5ea>

# feature preprocessing

Having features on a similar scale will help the gradient descent converge more quickly towards the minima

- ▶ NNs are very sensitive to feature preprocessing
  - If some of the features have a very large scale, then gradient descent takes more numbers of iterations for that feature to converge as compared to the other features having a small scale
  - If the feature which has high scale is highly correlated to the target output then the performance of the model will be affected as a whole because the weights for this particular feature will not converge to give the best performance.

<https://medium.com/@piyush.kailash.chaudhari/importance-of-feature-scaling-for-artificial-neural-networks-and-k-nearest-neighbors-4b7aa618d5ea>

# Gradient Descent

## ► Three types of GD

- Vanilla version (batch GD)) uses all of the training dataset to perform one update : – slow, intractable if dataset doesn't fit in memory. It is guaranteed to converge to global minimum for convex function and to local minimum for non-convex function.
- SGD version conducts the parameter update for each training example : much faster. Has difficulty to converge to global minimum, but this can be solved by slowly decreasing the learning rate.

# Gradient Descent

- mini-batch gradient descent:
  - Update is done for every  $n$  training examples
  - Leads to more stable convergence, but does not guarantee good convergence

# minibatch gradient descent: pseudocode

initialize  $w$

repeat ...

select a small batch (subset)  $X_b, Y_b$  from the  $X, Y$

compute gradient  $\nabla f_b$  of the loss for current batch  $X_b, Y_b$

$w = w - \eta \cdot \nabla f_b(w)$

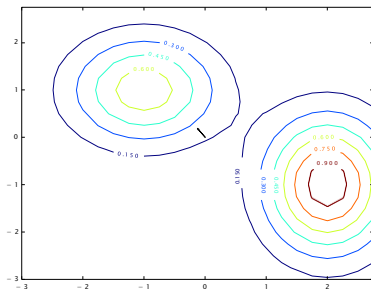
return  $w$

► in Keras:

`model.fit(X, Y, batch_size=400)`

# optimizing NNs

- ▶ unlike the linear classifiers we studied previously, NNs have non-convex objective functions with a lot of local minima
- ▶ so the end result depends on initialization

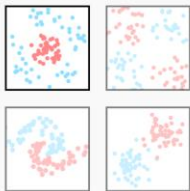


- ▶ when working with NNs, it is good practice to re-run with different initializations (e.g. different random seeds)

# optimizing NNs

## 1. Choose input dataset

Select a training dataset.



This legend details the color scheme for labels, and the values of the weights/gradients.



Node Type: ☐ Input ☒ ReLU ☐ Sigmoid

## 2. Choose initialization method

Select an initialization method for the values of your neural network parameters<sup>1</sup>.

☐ Zero ☒ Too small ☐ Appropriate ☐ Too large

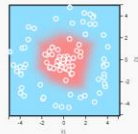
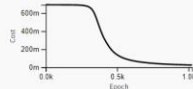


Select whether to visualize the weights or gradients of the network above.

☐ Weight ☒ Gradient

## 3. Train the network.

Observe the cost function and the decision boundary.

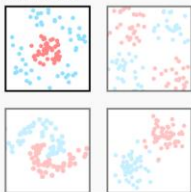


# optimizing NNs

Each time a neural network is initialized with a different set of weights, resulting in a different starting point for the optimization process, and potentially resulting in a different final set of weights with different performance characteristics.

## 1. Choose input dataset

Select a training dataset.



This legend details the color scheme for labels, and the values of the weights/gradients.



Node Type: Input ReLU Sigmoid

## 2. Choose initialization method

Select an initialization method for the values of your neural network parameters<sup>1</sup>.

☐ Zero ☐ Too small ☒ Appropriate ☐ Too large

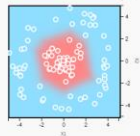
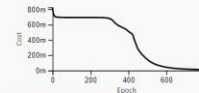


Select whether to visualize the weights or gradients of the network above.

☐ Weight ☒ Gradient

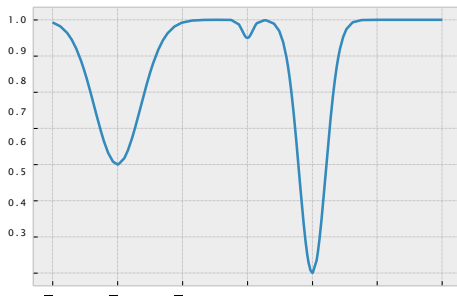
## 3. Train the network.

Observe the cost function and the decision boundary.



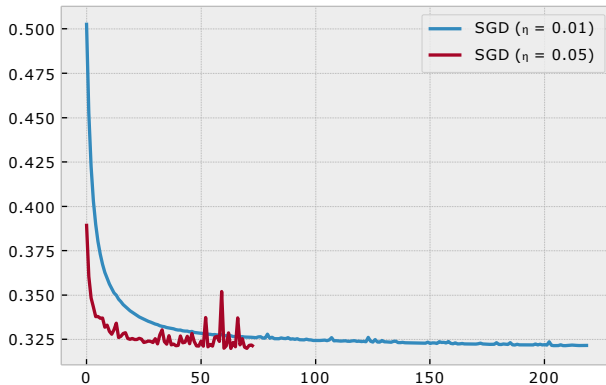
# plateaus in the objective

- ▶ NN objectives tend to have plateaus:



- ▶ this irregular shape makes it hard to set the learning rate

## example: two different learning rates



# adaptive gradient updates

- ▶ **adaptive** gradient descent methods control  $\eta$  to accelerate and slow down when necessary
- ▶ popular adaptive methods: **Adam, Adagrad, RMSProp**, ...
- ▶ in Keras:  

```
model.compile(..., optimizer='adam', ...)
```
- ▶ see Sebastian Ruder's report [An overview of gradient descent optimization algorithms](#) for an overview

# avoiding overfitting

- ▶ we've already seen how to apply a **regularizer** for logistic regression and SVC

$$\sum \text{Loss}(x_i, y_i, \mathbf{w}) + R(\mathbf{w})$$

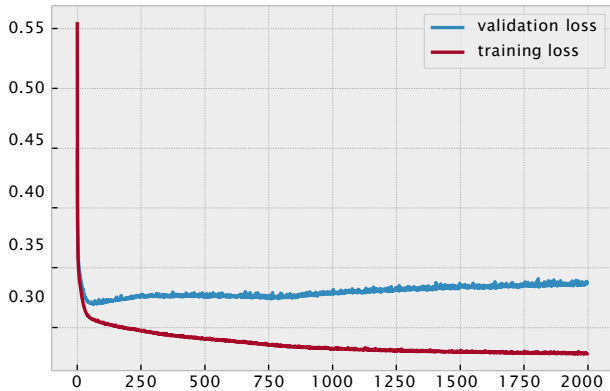
where  $R(\mathbf{w})$  can be  $\|\mathbf{w}\|^2$ , for instance

- ▶ in Keras:

```
from keras import regularizers  
regularizer = regularizers.l2(0.001)
```

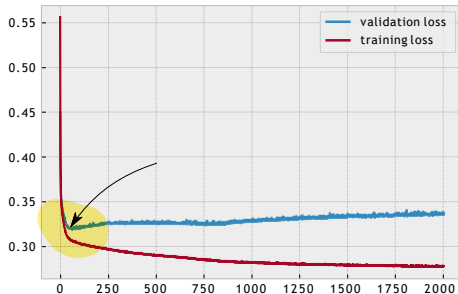
- ▶ in the neural network world, there are a few other methods that are also popular, such as **early stopping** and **dropout**

## example



# early stopping

- ▶ reserve a held-out development set (validation set)
- ▶ terminate training when there is no improvement on the held-out data

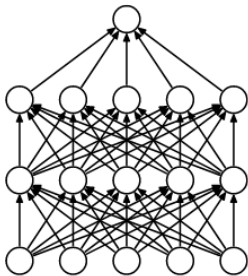


# early stopping: Keras

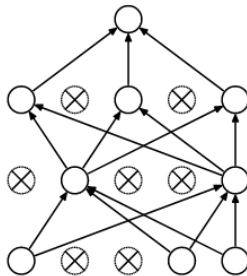
```
from keras import callbacks  
  
cb = callbacks.EarlyStopping(monitor='val_loss', min_delta=0,  
                             patience=10, verbose=0, mode='auto' )  
  
...  
  
model.fit(X, Y, epochs=1000, batch_size=400, verbose=2,  
          validation_split=0.1, callbacks=[cb])
```

[https://www.tensorflow.org/api\\_docs/python/tf/keras/callbacks/EarlyStopping](https://www.tensorflow.org/api_docs/python/tf/keras/callbacks/EarlyStopping)

# dropout



(a) Standard Neural Net

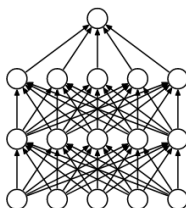


(b) After applying dropout.

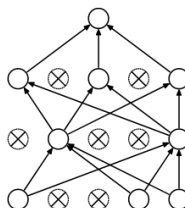
[\[source\]](#)

# dropout in neural networks

- ▶ **dropout** is often used to **reduce the risk of overfitting** in neural networks



(a) Standard Neural Net



(b) After applying dropout.

- ▶ why does it work?
- ▶ [Srivastava et al. \(2014\)](#) motivate dropout in terms of ensembles
  - ▶ if there are  $N$  connections in the model, we can see it as an ensemble of  $2^N$  different models (subsets of connections)

[source]

# dropout: Keras

```
from keras.layers import Dropout

model = Sequential()
model.add(Dense(... something ...))
model.add(Dropout(0.1))
model.add(Dense(... something ...))

model.compile(...)
model.fit(...)
```

noising, data augmentation, ...

- ▶ to reduce overfitting, we can add some random **noise** to training instances
- ▶ more generally, we can apply **data augmentation** to expose our model to more variation during training
  - ▶ for instance, blurring or rotating images

# summary

- ▶ for NN development, we will use libraries such as Keras, PyTorch or TensorFlow
- ▶ practical things to keep in mind:
  - ▶ **scaling** numerical features
  - ▶ **minibatching**: processing instances in parallel
  - ▶ **optimization**: SGD and its adaptive variants
  - ▶ regularization including **early stopping**, **dropout**, and **noising** to avoid overfitting

## Question 2 of 12: Training a neural network (3 points)

We use Keras to train a neural network for a binary classification task. The diagnostic output printed by Keras during training (100 epochs) is shown below.

```
Epoch 5: loss: 0.3206 - acc: 0.8499 - val_loss: 0.3289 - val_acc: 0.8460
Epoch 10: loss: 0.2962 - acc: 0.8614 - val_loss: 0.3219 - val_acc: 0.8477
Epoch 15: loss: 0.2806 - acc: 0.8695 - val_loss: 0.3216 - val_acc: 0.8502
Epoch 20: loss: 0.2681 - acc: 0.8742 - val_loss: 0.3284 - val_acc: 0.8502
Epoch 25: loss: 0.2557 - acc: 0.8812 - val_loss: 0.3311 - val_acc: 0.8498
Epoch 30: loss: 0.2448 - acc: 0.8856 - val_loss: 0.3415 - val_acc: 0.8484
Epoch 35: loss: 0.2345 - acc: 0.8907 - val_loss: 0.3516 - val_acc: 0.8465
Epoch 40: loss: 0.2250 - acc: 0.8942 - val_loss: 0.3608 - val_acc: 0.8456
Epoch 45: loss: 0.2168 - acc: 0.8985 - val_loss: 0.3766 - val_acc: 0.8428
Epoch 50: loss: 0.2098 - acc: 0.9015 - val_loss: 0.3846 - val_acc: 0.8394
Epoch 55: loss: 0.2023 - acc: 0.9052 - val_loss: 0.3946 - val_acc: 0.8380
Epoch 60: loss: 0.1950 - acc: 0.9087 - val_loss: 0.4110 - val_acc: 0.8383
Epoch 65: loss: 0.1893 - acc: 0.9113 - val_loss: 0.4182 - val_acc: 0.8386

Epoch 70: loss: 0.1827 - acc: 0.9147 - val_loss: 0.4317 - val_acc: 0.8360
Epoch 75: loss: 0.1778 - acc: 0.9175 - val_loss: 0.4427 - val_acc: 0.8338
Epoch 80: loss: 0.1714 - acc: 0.9200 - val_loss: 0.4528 - val_acc: 0.8322
Epoch 85: loss: 0.1671 - acc: 0.9222 - val_loss: 0.4641 - val_acc: 0.8322
Epoch 90: loss: 0.1626 - acc: 0.9242 - val_loss: 0.4729 - val_acc: 0.8372
Epoch 95: loss: 0.1580 - acc: 0.9263 - val_loss: 0.4855 - val_acc: 0.8317
Epoch 100: loss: 0.1538 - acc: 0.9286 - val_loss: 0.4934 - val_acc: 0.8327
```

After training, we evaluate the classifier on a separate test set and get an accuracy of 0.82.

CHALM How do you think we can improve the test set accuracy somewhat?